ANO 1 · PERÍODO 1 · MÓDULO 3 · SEMANA 1

# Introdução à gestão da engenharia de software

O software é construído por equipes, muda constantemente e precisa ser rastreável. Nesta semana você aprende como o desenvolvimento é organizado como um processo, como a gestão de configuração mantém o produto sob controle e como o Git e o GitHub permitem que pessoas trabalhem no mesmo código sem perder as mudanças umas das outras.

Guia de estudo | 8 lições | 600 min

SE Study Lab · Material de estudo

# Conteúdo

1. O que é engenharia de software
2. O ciclo de vida de desenvolvimento e os modelos de processo
3. Engenharia de requisitos: de desejos a especificações
4. Modelar software: de diagramas de contexto à UML
5. Gestão de configuração de software
6. Sistemas de controle de versões
7. Fundamentos do Git: áreas, commits e branches
8. GitHub, remotos e fluxos de colaboração
9. Soluções comentadas
10. Glossário
11. Questionário de prática

### Objetivos de aprendizagem

- Explicar a diferença entre programar e fazer engenharia de software.
- Descrever um ciclo de vida de desenvolvimento de software e dois modelos de processo.
- Escrever requisitos verificáveis e distinguir os funcionais dos não funcionais.
- Ler e esboçar modelos básicos: diagrama de contexto, modelo ER e máquina de estados.
- Definir a gestão de configuração de software e suas principais atividades.
- Usar comandos básicos do Git para inicializar, preparar, fazer commit, criar branches e fazer merge.
- Conectar um repositório local a um remoto e descrever um fluxo com pull requests.

# 01 Lição 1 · 15 min
# O que é engenharia de software

Programar é escrever instruções que um computador pode executar. A engenharia de software acrescenta a disciplina necessária para entregar um sistema que funcione, que possa ser alterado e que seja compreendido pelas pessoas que dependem dele. A diferença importa porque a maioria das falhas de software não são erros de digitação. Elas vêm de requisitos pouco claros, planejamento ruim, documentação ausente e falhas de comunicação entre o cliente e quem constrói.

Uma forma útil de ver essa lacuna é uma cadeia de traduções. O cliente tem uma necessidade real. Ele a descreve de forma imperfeita. As vendas prometem algo. Os analistas escrevem requisitos. Os designers propõem uma estrutura. Os programadores escrevem código. Cada passagem pode perder significado. A engenharia de software tenta tornar cada passagem explícita e verificável, com especificações escritas, revisões e testes.

O software também é um produto especial. Ele é intangível, muda com frequência e raramente está terminado. Por isso, os engenheiros trabalham com modelos: descrições simplificadas de um sistema, como um diagrama do fluxo de dados ou uma lista de casos de uso. Um bom modelo é abstrato o bastante para ser simples, preciso o bastante para ser confiável e barato o bastante para ser alterado antes de escrever código.

**Pense assim**

Construir uma casa exige plantas antes dos tijolos. O software precisa de planos e modelos antes do código, porque mudar uma parede no papel custa muito menos do que derrubá-la.

**Ideias-chave**

- A engenharia de software é uma disciplina sobre entregar e manter software, não apenas escrevê-lo.
- A maioria dos problemas de projeto vem de falhas de comunicação e de planejamento.
- Os modelos permitem que as equipes discutam e verifiquem um sistema antes de construí-lo.

## Erros comuns

- ✗ Tratar "compila" como "atende à necessidade".
  ✓ **Solução:** Compare o resultado com requisitos escritos e testes.

## Exercícios

**1.1 Encontre a lacuna** Nível 1

Um cliente diz "deixe o relatório mais rápido". Liste duas perguntas que você faria antes de escrever qualquer código.

*Dica: Pergunte o que "mais rápido" significa para ele: segundos, minutos ou outro conjunto de dados?*

**1.2 Escreva um requisito testável** Nível 2

Reescreva "o login deve ser seguro" como um requisito com uma condição mensurável.

*Dica: Nomeie a condição, o ator e o resultado esperado.*

Lição 2 · 20 min

# 02 O ciclo de vida de desenvolvimento e os modelos de processo

Um processo de desenvolvimento de software define o que é feito, por quem, quando e como. Todo processo realiza quatro atividades amplas: especificar o software, desenvolvê-lo, validá-lo e evoluí-lo após a entrega. O ciclo de vida é a sequência de fases pelas quais um produto passa: análise de requisitos, design, implementação, testes, implantação e manutenção.

Um processo em cascata executa essas fases uma única vez, em ordem. É fácil de planejar, mas lento para reagir a mudanças, e problemas descobertos nos testes custam caro para corrigir. Um processo iterativo e incremental divide o projeto em iterações. Cada iteração planeja, analisa, projeta, constrói e testa uma pequena parte, e produz uma linha de base: uma versão funcional e parcialmente completa do sistema. A diferença entre duas linhas de base é um incremento.

Processos do tipo unificado organizam as iterações em fases: concepção (objetivos e escopo), elaboração (arquitetura e principais riscos), construção (construção do sistema) e transição (entrega). Métodos adaptativos como o Scrum aplicam a mesma ideia com ciclos curtos e feedback frequente dos usuários. Em todos os casos, a pergunta-chave é quão cedo a equipe aprende algo que seria caro aprender tarde.

**Pense assim**

Pense em uma aula de culinária. Cascata é seguir uma receita do início ao fim sem provar. O trabalho iterativo é provar o molho depois de cada passo e ajustar o próximo.




Diagrama · O ciclo de vida de desenvolvimento e os modelos de processo

**Ideias-chave**

- Um modelo de processo é uma descrição simplificada de como equipes reais trabalham.
- O trabalho iterativo produz linhas de base; a diferença entre linhas de base é um incremento.
- Escolha um processo pelo quanto de mudança você espera e pelo custo dos erros tardios.

## Erros comuns

- ✗ Chamar um projeto em cascata de "ágil" porque ele tem reuniões.
  ✓ **Solução:** Pergunte se o feedback muda o plano durante o projeto.

## Exercícios

**2.1 Escolha um modelo** Nível 1

Um formulário do governo tem regras legais fixas que não vão mudar por dois anos. Qual modelo de processo se encaixa melhor, e por quê?

*Dica: Pense em quanta mudança você espera depois que os requisitos forem assinados.*

**2.2 Desenhe seu próprio histórico de linhas de base** Nível 2

Uma equipe constrói um app de chat em três iterações: login, mensagens e notificações. Nomeie a linha de base e o incremento após a iteração 2.

*Dica: Uma linha de base é uma versão funcional; o incremento é a diferença em relação à anterior.*

Lição 3 · 25 min

# 03 Engenharia de requisitos: de desejos a especificações

Um requisito é uma capacidade ou condição que um sistema precisa ter para ser aceito por seus usuários, pelo cliente ou por uma norma. A engenharia de requisitos transforma um desejo de negócio em afirmações precisas e verificáveis que analistas, designers, programadores e testadores compartilham. Erros aqui são os mais caros, porque todas as fases seguintes são construídas sobre eles.

O trabalho passa por sete atividades que muitas vezes se sobrepõem. A concepção identifica a necessidade e uma solução de alto nível. A elicitação reúne as necessidades das partes interessadas. A elaboração as refina em cenários, regras de negócio e fluxos principais e alternativos. A negociação concilia visões conflitantes e pondera risco, custo e prazo. A especificação registra tudo em um modelo padrão, muitas vezes com diagramas e protótipos. A validação revisa a especificação com os usuários e corrige problemas antes de construir. A gestão acompanha cada mudança nos requisitos durante todo o projeto.

A elicitação é difícil por três motivos. Problemas de escopo: os limites do sistema não estão claros, então o analista escreve premissas e restrições que dizem o que entra e o que fica de fora. Problemas de compreensão: os usuários podem não saber do que precisam ou descrevê-lo em um nível alto demais. Volatilidade: os requisitos mudam de uma reunião para outra, então as equipes registram os acordos em atas e trabalham com um usuário representante.

Os requisitos vêm em camadas. Os requisitos de negócio expressam os objetivos da organização. Os requisitos das partes interessadas descrevem as necessidades de um grupo de usuários. Os requisitos da solução dizem o que o sistema deve fazer e se dividem em funcionais (comportamento: "o sistema envia um recibo por e-mail após o pagamento") e não funcionais (qualidades: "uma página carrega em menos de 2 segundos com 500 usuários conectados"). Os requisitos de transição são capacidades temporárias para passar da situação antiga para a nova, como migrar dados antigos.

> **Pense assim**
> Um alfaiate mede duas vezes e corta uma. Os requisitos são as medidas: se estiverem errados, nenhuma costura cuidadosa (programação) fará o terno servir.




Diagrama · Engenharia de requisitos: de desejos a especificações

**Ideias-chave**

- Um requisito precisa ser verificável: alguém deve poder dizer "atendido" ou "não atendido".
- Sete atividades: concepção, elicitação, elaboração, negociação, especificação, validação e gestão.
- Funcional = o que o sistema faz. Não funcional = quão bem ele faz.

## Erros comuns

- ✗ Escrever "o sistema deve ser rápido e fácil".
  ✓ **Solução:** Acrescente uma medida: "os resultados da busca aparecem em menos de 1 segundo".
- ✗ Mudar requisitos pelo chat sem registro.
  ✓ **Solução:** Registre cada acordo e quem o aprovou; isso é gestão de requisitos.

## Exercícios

**3.1 Encontre o limite** Nível 1

Uma biblioteca pede "um sistema para livros". Escreva duas premissas ou restrições que definam o escopo.

*Dica: Diga o que está incluído e o que fica explicitamente de fora.*

**3.2 Separe as camadas** Nível 2

Para uma farmácia on-line, escreva um requisito de negócio, um funcional, um não funcional e um de transição.

*Dica: Negócio = objetivo; funcional = ação; não funcional = qualidade com um número; transição = temporário.*

Lição 4 · 25 min

# 04 Modelar software: de diagramas de contexto à UML

Um modelo é uma representação simplificada de um sistema que ajuda você a raciocinar sobre ele. Bons modelos têm cinco qualidades: são abstratos (deixam de fora o que não importa), compreensíveis, precisos (fiéis ao sistema), preditivos (permitem conclusões corretas) e mais baratos do que construir e estudar o sistema real. Os modelos se tornam a linguagem comum entre usuários, analistas e desenvolvedores.

Os modelos de análise descrevem o problema no mundo real: o que o negócio faz. Os modelos de design descrevem a solução de software: como o sistema vai fazê-lo. A engenharia direta vai de um modelo ao código funcionando; a engenharia reversa reconstrói um modelo a partir de um produto ou código existente, algo comum quando falta documentação.

No início de um projeto, um diagrama de contexto desenha o sistema como uma única caixa cercada pelos atores externos e pelos dados que entram e saem; ele fixa os limites. Depois, um diagrama de fluxo de dados divide o processo em etapas com entradas e saídas. Para os dados, um modelo entidade-relacionamento mostra entidades, seus atributos e relacionamentos com cardinalidade: um para um, um para muitos ou muitos para muitos. Em uma tabela, a chave primária identifica cada linha, e uma chave estrangeira guarda a chave primária de outra tabela para ligá-las. Uma máquina de estados mostra os estados em que um componente pode estar, como Rascunho, Pago e Enviado, e os eventos que o fazem mudar. Um dicionário de dados define cada item de dado, para que todos usem o mesmo significado.

A UML, a Linguagem de Modelagem Unificada, padroniza diagramas para o desenvolvimento orientado a objetos. Os diagramas de estrutura mostram as partes de um sistema: classes, componentes, objetos, pacotes e implantação. Os diagramas de comportamento e interação mostram como ele age: casos de uso, atividades, estados e sequência. Raramente você precisa de todos. Escolha o diagrama que responde à pergunta que sua equipe está fazendo agora.

> **Pense assim**
> O mapa de uma cidade é um modelo. Ele ignora a cor de cada casa (abstrato), mas acerta as ruas (preciso), então você pode planejar uma rota antes de caminhar (preditivo e barato).

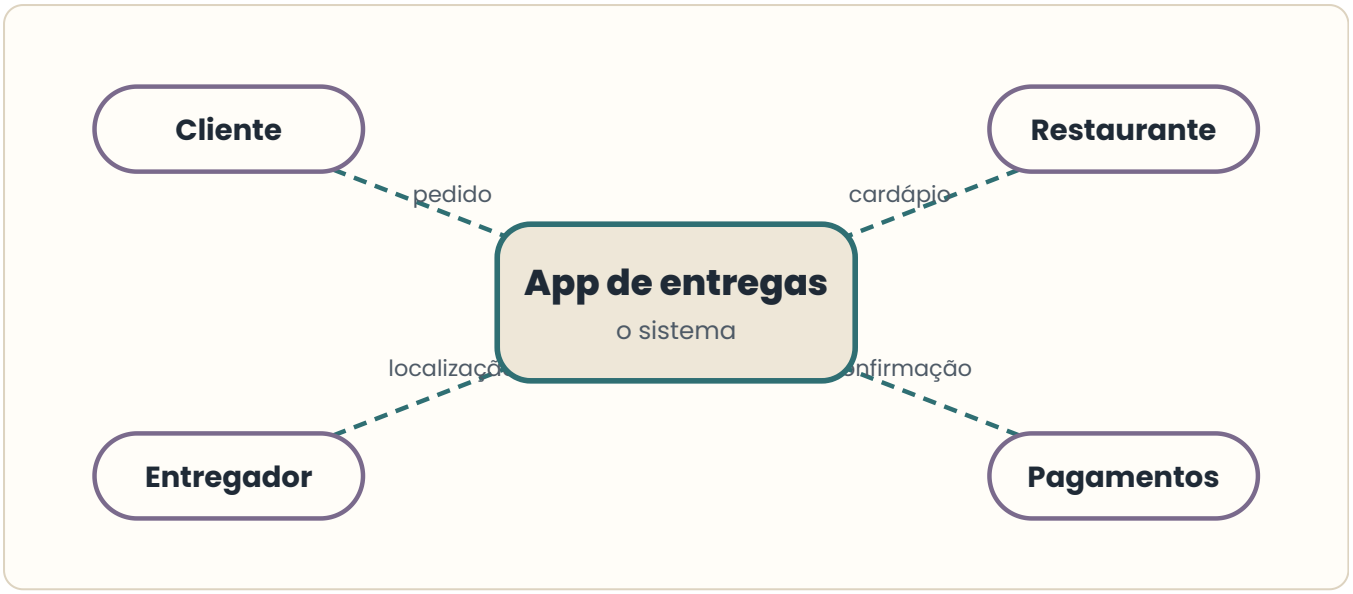



Diagrama · Modelar software: de diagramas de contexto à UML

**Ideias-chave**

- Um modelo é útil quando é mais simples que o sistema, mas continua correto no que importa.
- Diagrama de contexto = limites; modelo ER = dados; máquina de estados = ciclo de vida de uma coisa.
- A UML tem diagramas de estrutura e de comportamento; escolha pela pergunta que você precisa responder.

## Erros comuns

- ✗ Desenhar todos os diagramas UML "para ficar completo".
  ✓ **Solução:** Desenhe o diagrama que responde à pergunta atual.

## Exercícios

**4.1 Desenhe um diagrama de contexto com palavras** Nível 1

Para um app de entrega de comida, liste os atores externos e um fluxo de dados de entrada e de saída para cada um.

*Dica: Pense em clientes, restaurantes, entregadores e no provedor de pagamento.*

**4.2 Projete uma máquina de estados** Nível 3

Liste os estados de um pedido on-line e o evento de cada transição.

*Dica: Comece em Criado; termine em Entregue ou Cancelado.*

# 05 Lição 5 · 15 min
# Gestão de configuração de software

A gestão de configuração de software (GCS) controla como as partes de um sistema mudam ao longo do tempo. Seus itens não são apenas código-fonte: documentos de requisitos, designs, scripts de teste, arquivos de build e manuais são todos itens de configuração. O objetivo é saber exatamente qual versão de cada item pertence a qual versão do produto.

A GCS se apoia em quatro ideias. A identificação nomeia cada item e versão. O controle de versões registra cada mudança. O controle de mudanças decide quais mudanças são aprovadas e quando. A contabilidade de status informa o que está atualmente em cada linha de base. Uma auditoria então verifica se a linha de base corresponde ao que se pretendia.

Sem GCS, as equipes sofrem com perguntas como "qual build está em produção?" ou "por que este teste passou ontem?". Manter uma linha de base estável permite que as pessoas continuem trabalhando em paralelo sem sobrescrever o trabalho umas das outras.

**Pense assim**

Uma biblioteca mantém um catálogo de cada edição e de quem a emprestou. A GCS faz o mesmo com um produto: cada item, versão e mudança tem um registro que você pode consultar.

**Ideias-chave**

- Os itens de configuração incluem tudo o que compõe o produto, não apenas o código.
- A GCS responde: qual versão, qual mudança, aprovada por quem e em qual linha de base.

## Exercícios

**5.1 Faça o inventário dos itens de configuração** Nível 1

Liste cinco itens de configuração de um app móvel que não sejam código-fonte.

*Dica: Pense em documentos, testes, configurações de build e versões.*

**5.2 Rastreie um bug até uma linha de base** Nível 3

A produção mostra um bug que não estava no build de teste. Descreva as quatro perguntas da GCS que você responderia para encontrar a causa.

*Dica: Identificação, controle de versões, controle de mudanças, contabilidade de status.*

06 Lição 6 · 20 min

# Sistemas de controle de versões

Um sistema de controle de versões (VCS) registra mudanças em arquivos ao longo do tempo. Ele oferece um histórico do que mudou, quando e por quem; a possibilidade de recuperar versões anteriores; e uma forma de várias pessoas trabalharem no mesmo projeto sem perder o trabalho umas das outras.

Sistemas centralizados mantêm um repositório principal em um servidor. Sistemas distribuídos, como o Git, dão a cada desenvolvedor uma cópia local completa do repositório e de seu histórico. Você pode fazer commits, criar branches e navegar pelo histórico sem conexão, e compartilhar mudanças quando estiver conectado. O Git é um VCS distribuído. GitHub, GitLab e serviços semelhantes hospedam repositórios Git e acrescentam recursos de colaboração; eles não são o Git em si.

Um commit é uma fotografia do projeto com uma mensagem. Bons commits representam uma única mudança lógica, e suas mensagens dizem o que mudou no modo imperativo, por exemplo "add sum functionality" ou "fix off-by-one in pagination". Commits pequenos e frequentes tornam o histórico mais fácil de ler e de reverter.

**Pense assim**

O controle de versões é como um videogame com pontos de salvamento. Você pode voltar a qualquer ponto salvo, ver o que mudou entre dois deles e deixar que seus amigos joguem a partir da própria cópia.

**Ideias-chave**

- Um VCS guarda o histórico, permite recuperar versões e mostra quem mudou o quê e quando.
- O Git é distribuído; o GitHub é um serviço de hospedagem de repositórios Git.
- Um commit deve conter uma única mudança lógica com uma mensagem clara.

## Erros comuns

- ✗ Escrever mensagens de commit como "coisas" ou "mudanças".
  ✓ **Solução:** Descreva a mudança em uma linha imperativa de cerca de 50 caracteres.

## Exercícios

**6.1 Corrija a mensagem de commit** Nível 1

Reescreva estas mensagens no estilo imperativo: "stuff", "Fixed the bug.", "Added a new function for sums".

*Dica: Comece com um verbo, em minúsculas, sem ponto final, com cerca de 50 caracteres.*

**6.2 Divida um commit** Nível 2

Um commit mistura uma correção de bug, uma renomeação e um teste novo. Descreva como você o dividiria em três commits lógicos.

*Dica: Prepare juntos os arquivos relacionados e faça o commit de cada grupo por vez.*

07 Lição 7 · 30 min

# Fundamentos do Git: áreas, commits e branches

O Git tem três áreas. O diretório de trabalho contém os arquivos que você edita. A área de preparação (também chamada de índice) contém as mudanças que você escolheu para o próximo commit. O repositório, guardado na pasta oculta .git, contém o histórico confirmado. A maioria dos comandos move mudanças entre essas áreas: git add leva o trabalho para a preparação, e git commit leva o que foi preparado para o repositório.

O ciclo do dia a dia é: editar, conferir, preparar, fazer commit. Rode git status com frequência; ele mostra quais arquivos foram modificados, quais estão preparados e quais não são rastreados. Use git log --oneline --graph para ler o histórico. Use git diff para ver as mudanças não preparadas e git diff --staged para ver o que o próximo commit vai conter.

Uma branch é um ponteiro móvel para um commit. Criar uma branch não copia arquivos. Ela permite desenvolver uma funcionalidade sem mudar a main. Quando a funcionalidade está pronta, você faz o merge de volta. Se as duas branches mudaram as mesmas linhas, o Git para e marca um conflito: o arquivo contém os marcadores <<<<<<<, ======= e >>>>>>> mostrando as duas versões. Você edita o arquivo até o resultado desejado, prepara e faz commit.

**Pense assim**

O Git é um estúdio fotográfico. O diretório de trabalho é o cenário, a preparação é escolher quais fotos vão para o álbum e o commit é imprimir a página do álbum. Uma branch é um segundo álbum que você pode montar sem mexer no primeiro.

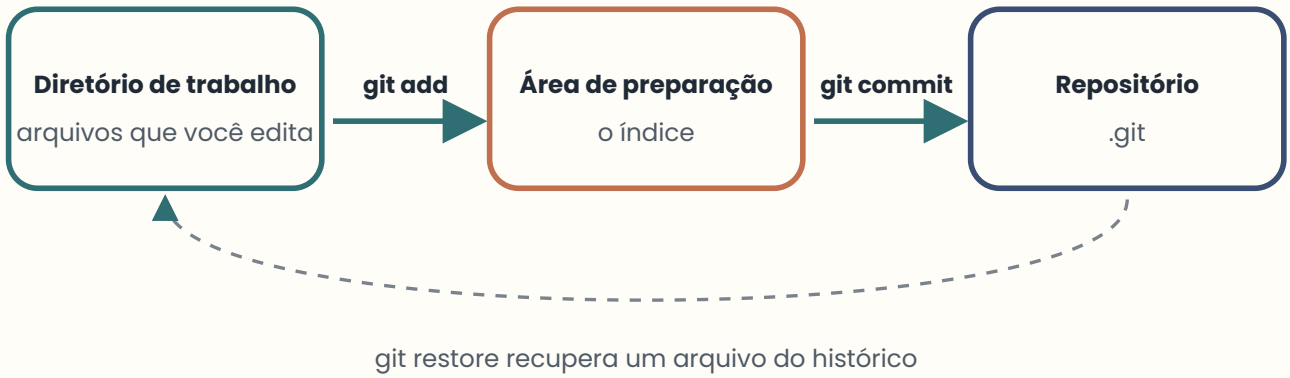



Diagrama · Fundamentos do Git: áreas, commits e branches

**Ideias-chave**

- Edite no diretório de trabalho, prepare com git add e registre com git commit.
- git status e git log mostram onde você está; use-os antes e depois de cada passo.
- Uma branch é um ponteiro. O merge combina históricos; conflitos exigem uma decisão humana.

## Erros comuns

- ✗ Fazer commit sem conferir o git status, incluindo arquivos que não têm relação.
  ✓ **Solução:** Rode git status e git diff --staged antes de cada commit.
- ✗ Apagar os marcadores de conflito sem escolher o conteúdo certo.
  ✓ **Solução:** Decida o conteúdo final das linhas, remova as três linhas de marcador e depois prepare e faça commit.

## Exercícios

**7.1 Preveja o status** Nível 1

Você edita a.txt, roda git add a.txt e depois edita a.txt de novo. A segunda edição está preparada? O que o git status mostra?

*Dica: A preparação captura o arquivo no momento em que você roda git add.*

**7.2 Leia o grafo** Nível 2

Rode git log --oneline --graph --all em uma pasta de teste após o Laboratório 2. Explique em uma frase por que main e feature-sum apontam para o mesmo commit.

*Dica: O que acontece quando a branch que recebe o merge não tem commits novos?*

**7.3 Resolva pela intenção** Nível 3

Duas branches mudaram a mesma função: uma renomeou um parâmetro e a outra acrescentou uma validação. Descreva a resolução desejada e os comandos.

*Dica: Mantenha as duas intenções, depois prepare e faça commit.*

# 08 Lição 8 · 25 min
# GitHub, remotos e fluxos de colaboração

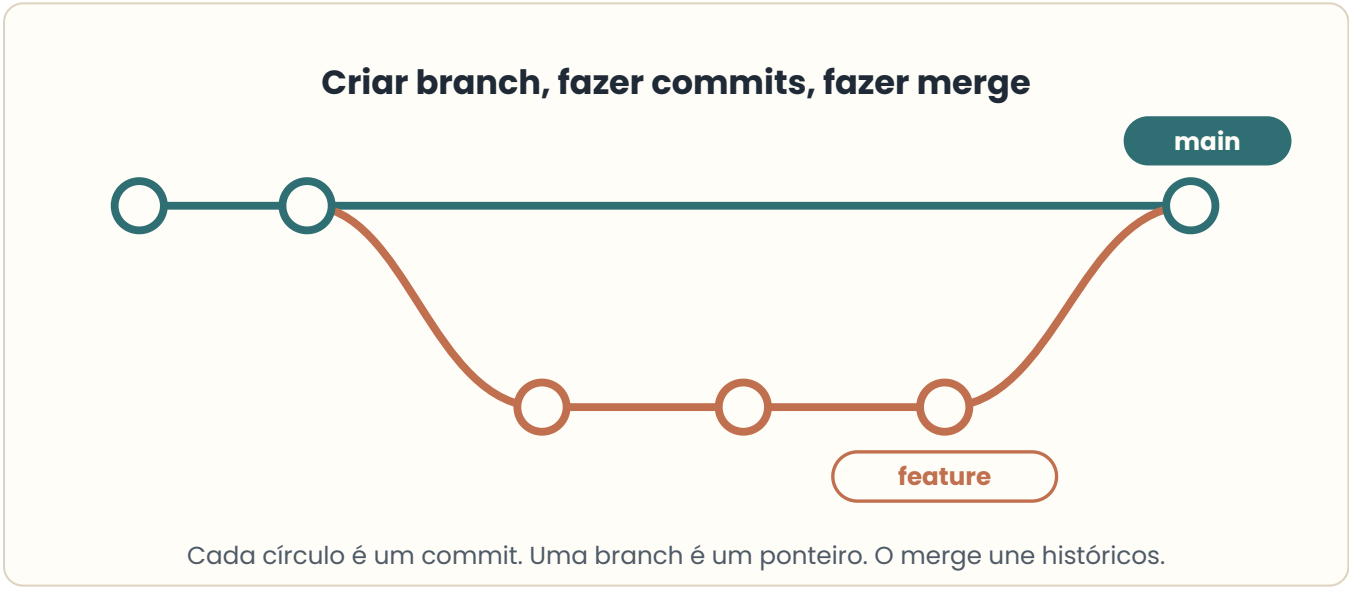
Um remoto é um link com nome para outro repositório, por convenção chamado origin. git remote add origin <url> o registra. git push -u origin main envia sua branch main e define o upstream. git fetch baixa commits novos sem mudar seus arquivos; git pull faz o fetch e depois os integra à sua branch atual.

Um fluxo comum em equipe usa branches de funcionalidade com pull requests (chamados de merge requests no GitLab). Você cria uma branch, envia-a e abre um pedido. Os revisores leem o diff, comentam e aprovam. Só depois da aprovação a branch é mesclada na main. A etapa de revisão é onde as equipes encontram defeitos e compartilham conhecimento.

Uma boa colaboração também significa manter a main estável, atualizar sua cópia local antes de começar a trabalhar e escrever commits que um revisor consiga entender um de cada vez.

**Pense assim**

Um pull request é como um rascunho de redação enviado a um colega antes da entrega. O revisor não escreve por você, mas encontra erros enquanto ainda são baratos de corrigir.




Cada círculo é um commit. Uma branch é um ponteiro. O merge une históricos.

Diagrama · GitHub, remotos e fluxos de colaboração

**Ideias-chave**

- origin é o nome convencional do remoto que você clonou ou conectou.
- fetch baixa; pull baixa e integra; push envia.
- Os pull requests tornam a revisão uma etapa planejada antes de o código chegar à main.

## Erros comuns

- ✗ Enviar diretamente para a main em um projeto compartilhado.
  ✓ **Solução:** Trabalhe em uma branch de funcionalidade e abra um pull request para revisão.

## Exercícios

**8.1 Ordene o fluxo de trabalho** Nível 1

Coloque em ordem: merge, abrir pull request, enviar a branch, revisar e aprovar, criar a branch.

*Dica: Você não pode abrir um pedido antes de a branch existir no remoto.*

**8.2 Recupere-se de um push rejeitado** Nível 3

Um colega enviou mudanças para a main enquanto você trabalhava. Seu git push foi rejeitado. Escreva a sequência segura de comandos.

*Dica: Primeiro fetch, depois integre e só então push. Nunca force.*

# Soluções comentadas

**1.1 Encontre a lacuna.** Exemplo: "Qual relatório, e quão lento ele está hoje em relação à meta?" e "Quem o usa, e com que frequência?" Cada resposta transforma um desejo vago em um requisito que você pode testar.

**1.2 Escreva um requisito testável.** Exemplo: "Após 5 tentativas de login falhas em 10 minutos, a conta é bloqueada por 15 minutos e o usuário recebe um e-mail." Agora um testador consegue verificá-lo.

**2.1 Escolha um modelo.** Um processo sequencial (do tipo cascata) pode servir, porque os requisitos são estáveis e erros iniciais seriam caros. Um processo iterativo também funcionaria se a equipe quiser feedback cedo sobre a interface.

**2.2 Desenhe seu próprio histórico de linhas de base.** A linha de base 2 é login mais mensagens, funcionando e documentada. O incremento é a funcionalidade de mensagens em comparação com a linha de base 1.

**3.1 Encontre o limite.** Exemplo: "Inclui empréstimos e devoluções para sócios." "Exclui a compra de livros novos e o pagamento de salários." Limites claros evitam problemas de escopo.

**3.2 Separe as camadas.** Negócio: "aumentar as vendas on-line em 20% este ano." Funcional: "os clientes enviam uma foto da receita." Não funcional: "os pedidos são confirmados em até 5 segundos." Transição: "importar 10 anos de registros de clientes do sistema antigo."

**4.1 Desenhe um diagrama de contexto com palavras.** Cliente: pedido entra, status sai. Restaurante: cardápio entra, pedido sai. Entregador: localização entra, tarefa de coleta sai. Provedor de pagamento: confirmação entra, solicitação de cobrança sai.

**4.2 Projete uma máquina de estados.** Criado --pagar--> Pago --embalar--> Embalado --enviar--> Enviado --entregar--> Entregue. Criado ou Pago --cancelar--> Cancelado. Cada seta é um evento; cada caixa é um estado.

**5.1 Faça o inventário dos itens de configuração.** Exemplos: especificação de requisitos, arquivos de design da interface, plano e scripts de teste, arquivos de configuração de build, notas de versão, manual do usuário.

**5.2 Rastreie um bug até uma linha de base.** Quais versões exatas estão em produção (identificação)? O que mudou entre o build de teste e a produção (controle de versões)? Quais mudanças foram aprovadas e por quem (controle de mudanças)? O build de produção é o registrado na linha de base (contabilidade de status)?

**6.1 Corrija a mensagem de commit.** fix login redirect loop | fix bug where … (nomeie o bug) | add sum function. Verbos específicos ajudam os revisores a percorrer o histórico rapidamente.

**6.2 Divida um commit.** Prepare os arquivos da correção e faça commit; prepare os arquivos renomeados e faça commit; prepare o teste e faça commit. Agora cada commit pode ser revisado ou revertido sozinho.

**7.1 Preveja o status.** Não: a segunda edição não está preparada. O git status lista a.txt tanto em "Changes to be committed" (a primeira versão) quanto em "Changes not staged for commit" (a edição mais nova).

**7.2 Leia o grafo.** A main não tinha commits novos desde que a feature-sum se separou, então o Git moveu o ponteiro para a frente (fast-forward) em vez de criar um commit de merge.

**7.3 Resolva pela intenção.** Edite a função para que use o novo nome do parâmetro e inclua a validação. Remova todos os marcadores, rode git add no arquivo e depois git commit para concluir o merge.

**8.1 Ordene o fluxo de trabalho.** Criar a branch → enviar a branch → abrir o pull request → revisar e aprovar → fazer o merge.

**8.2 Recupere-se de um push rejeitado.** git fetch origin, depois git pull origin main (ou git rebase origin/main), resolva os conflitos, rode os testes e então git push origin main.

# Glossário

| Pergunta | Resposta |
| --- | --- |
| Qual é a principal diferença entre programar e fazer engenharia de software? | Programar é escrever código. A engenharia de software também planeja, especifica, valida e mantém um sistema que atende a uma necessidade real. |
| Por que os engenheiros constroem modelos antes de escrever código? | Modelos são mais baratos de mudar do que código. Eles simplificam o sistema, facilitam a discussão e permitem prever o comportamento. |
| O que um processo de desenvolvimento de software define? | O que é feito, por quem, quando e como. Suas quatro atividades comuns são especificação, desenvolvimento, validação e evolução. |
| Cascata vs. iterativo e incremental: qual é a diferença principal? | A cascata executa cada fase uma vez, em ordem. O trabalho iterativo repete as fases em ciclos curtos e entrega incrementos funcionando. |
| O que é uma linha de base? | Uma versão acordada e guardada do sistema e de seus documentos. O trabalho posterior é medido em relação a ela. |
| O que é um incremento? | A diferença entre duas linhas de base sucessivas. |
| Cite três tipos de item de configuração além do código-fonte. | Quaisquer três entre: documentos de requisitos, designs, scripts de teste, arquivos de build, manuais. |
| Controle de mudanças vs. contabilidade de status | O controle de mudanças decide quais mudanças são aprovadas. A contabilidade de status informa o que está atualmente em cada linha de base. |
| Dê dois motivos para usar um sistema de controle de versões. | Mantém o histórico, recupera versões anteriores, mostra quem mudou o quê e quando, e permite trabalho em paralelo. |
| Controle de versões centralizado vs. distribuído | Centralizado: um repositório principal em um servidor. Distribuído: cada desenvolvedor tem uma cópia local completa com o histórico, como no Git. |
| Git vs. GitHub | O Git é a ferramenta de controle de versões. O GitHub é um serviço de hospedagem de repositórios Git com recursos de revisão e colaboração. |
| O que é um bom commit? | Uma mudança lógica, pequena e frequente, com uma mensagem imperativa como "add sum functionality". |
| Cite as três áreas do Git. | Diretório de trabalho, área de preparação (índice) e repositório (.git). |
| O que o git add faz? | Move mudanças do diretório de trabalho para a área de preparação. Ele não cria um commit. |
| O que o git commit -m "mensagem" faz? | Registra as mudanças preparadas como um novo commit com a mensagem. |
| Por que rodar git status antes de fazer commit? | Ele mostra arquivos modificados, preparados e não rastreados, para que você não faça commit de mudanças sem relação. |

| | |
|---|---|
| O que o git log --oneline --graph mostra? | Um histórico de commits compacto com hashes curtos, mensagens e linhas das branches. |
| O que é uma branch no Git? | Um ponteiro móvel para um commit que permite desenvolver um trabalho de forma isolada. Criar uma não copia arquivos. |
| O que causa um conflito de merge? | Duas branches mudaram as mesmas linhas de formas diferentes. O Git não consegue decidir, então uma pessoa precisa escolher o resultado. |
| O que é origin? | O nome convencional de um repositório remoto, como a cópia do GitHub da qual você clonou. |
| git fetch vs. git pull | O fetch baixa commits remotos novos sem mudar seus arquivos. O pull faz o fetch e depois os integra à sua branch atual. |
| O que o git push -u origin main faz? | Envia sua branch main local para o origin e define o upstream, assim os próximos push e pull não precisam de argumentos. |
| Por que usar pull requests? | Eles tornam a revisão uma etapa planejada: os revisores leem o diff, comentam e aprovam antes de a branch ser mesclada na main. |
| Por que enviar diretamente para uma main compartilhada é arriscado? | Mudanças não testadas ou não revisadas chegam a todos de uma vez, e o histórico de branches compartilhadas não deve ser reescrito. |
| O que faz de uma afirmação um bom requisito? | É uma capacidade ou condição que o sistema deve atender, e alguém consegue verificar se ela é atendida. |
| Cite as sete atividades da engenharia de requisitos. | Concepção, elicitação, elaboração, negociação, especificação, validação e gestão. |
| Três motivos pelos quais a elicitação é difícil? | Escopo (limites pouco claros), compreensão (os usuários não sabem dizer do que precisam) e volatilidade (as necessidades continuam mudando). |
| Requisito funcional vs. não funcional | Funcional: um comportamento ("enviar um recibo"). Não funcional: uma qualidade com uma medida ("menos de 2 segundos"). |
| O que é um requisito de transição? | Uma capacidade temporária para passar do estado antigo para o novo, como migrar dados antigos. |
| Cinco qualidades de um bom modelo? | Abstrato, compreensível, preciso, preditivo e barato. |
| Modelo de análise vs. modelo de design | A análise descreve o problema do mundo real. O design descreve a solução de software. |
| O que um diagrama de contexto mostra? | O sistema como uma caixa, os atores externos ao redor e os dados que entram e saem. Ele fixa os limites. |
| Chave primária vs. chave estrangeira | A chave primária identifica cada linha de uma tabela. A chave estrangeira guarda a chave primária de outra tabela para ligar as duas. |

| | |
|---|---|
| Diagramas UML de estrutura vs. de comportamento | Estrutura: classes, componentes, pacotes, implantação. Comportamento: casos de uso, atividades, estados, sequência. |

# Questionário de prática

**1. Qual item é um item de configuração na gestão de configuração de software?**

A. Um documento de requisitos
B. A cafeteira do escritório
C. Uma agenda pessoal

**2. Qual afirmação sobre o Git está correta?**

A. O Git é um sistema de controle de versões distribuído; cada clone contém o histórico completo.
B. O Git não consegue registrar um commit sem uma conexão de internet com o GitHub.
C. Git e GitHub são dois nomes para a mesma ferramenta.

**3. Em um processo iterativo e incremental, o que representa a diferença entre duas linhas de base sucessivas?**

A. Um incremento
B. Um registro de riscos
C. Apenas a entrega final

**4. Verdadeiro ou falso: Git e GitHub são o mesmo produto.**

A. Verdadeiro
B. Falso

**5. Verdadeiro ou falso: o git add cria um commit no histórico do repositório.**

A. Verdadeiro
B. Falso

**6. Verdadeiro ou falso: criar uma branch no Git copia todos os arquivos do projeto.**

A. Verdadeiro
B. Falso

**7. As mudanças escolhidas para o próximo commit esperam na área de _____. (Dica: também chamada de índice.)**

*Resposta: _____*

**8. Digite um comando que crie uma branch chamada feature-login e mude para ela (sintaxe moderna).**

*Resposta: _____*

**9. Em duas frases, explique a diferença entre git fetch e git pull.**

*Resposta: _____*

**10. Cite um benefício dos commits pequenos e focados.**

*Resposta: _____*

**11. Associe cada comando à sua finalidade.**

*Associe cada item da esquerda a um da direita.*

| | |
|---|---|
| 1. git add | A. Mostra arquivos modificados, preparados e não rastreados |
| 2. git commit | B. Move mudanças para a área de preparação |
| 3. git status | C. Baixa commits remotos sem fazer merge |
| 4. git fetch | D. Registra as mudanças preparadas como um novo commit |

**12. Associe cada modelo de processo à sua característica principal.**

*Associe cada item da esquerda a um da direita.*

| | |
|---|---|
| 1. Cascata | A. Ciclos curtos com feedback frequente dos usuários |
| 2. Iterativo e incremental | B. As fases rodam uma vez, em ordem |
| 3. Adaptativo (por exemplo, Scrum) | C. Iterações repetidas que produzem linhas de base |

---

**13. Interprete os comandos. O que o git status informa sobre notes.txt?**

```
echo "v2" >> notes.txt
git add notes.txt
git status
```

A. notes.txt aparece em "Changes to be committed"
B. notes.txt já está no histórico do repositório
C. notes.txt não mudou

**14. Interprete o comando. O que a saída vai mostrar?**

```
git log --oneline -3
```

A. Os três commits mais recentes, um por linha, com hashes curtos e mensagens
B. Os três arquivos mais antigos do projeto
C. As três branches com mais conflitos

**15. Um colega diz que a última versão se comporta de forma incorreta. Qual pergunta você deve fazer primeiro?**

A. Quais itens de configuração e versões estão nessa linha de base?
B. Quem tem o notebook mais rápido?
C. Podemos apagar o repositório e começar de novo?

**16. Você quer que um colega revise sua mudança no login antes de chegar à main. O que você faz?**

A. Envio uma branch de funcionalidade e abro um pull request para revisão
B. Envio direto para a main e peço feedback depois
C. Mando por e-mail um zip do projeto

**17. Duas branches mudaram a mesma linha de app.js. O merge para. O que aconteceu?**

A. Um conflito de merge; uma pessoa precisa escolher o conteúdo final
B. O Git apagou o repositório
C. O remoto está fora do ar

**18. O git push é rejeitado porque o remoto tem commits que você não tem localmente. Qual é a correção mais segura?**

A. Rodar git fetch, integrar as mudanças remotas (merge ou rebase) e depois enviar de novo
B. Rodar git push --force para sobrescrever o remoto
C. Apagar seu repositório local e clonar de novo

**19. O git commit falha com "Author identity unknown". O que você deve fazer?**

A. Configurar seu nome e e-mail com git config --global user.name e user.email
B. Apagar a pasta .git
C. Fazer commit com git add -f

**20. Depois de um merge, um arquivo ainda contém linhas <<<<<<< HEAD. Qual é a resolução correta?**

A. Editar o arquivo com o conteúdo desejado, remover as linhas de marcador, fazer git add do arquivo e depois commit
B. Deixar os marcadores e fazer commit mesmo assim
C. Rodar git merge --abort depois do commit

**21. Qual afirmação é um requisito não funcional?**

A. Os resultados da busca aparecem em menos de um segundo
B. Os usuários podem redefinir a senha por e-mail
C. Os administradores podem exportar um relatório de vendas

**22. Verdadeiro ou falso: os requisitos de transição ficam no sistema para sempre.**

A. Verdadeiro
B. Falso

**23. A cada reunião, um usuário diferente muda o que foi combinado na semana anterior. Que problema de elicitação é este, e o que ajuda?**

A. Volatilidade: registrar os acordos em atas e trabalhar com um usuário representante
B. Escopo: comprar um servidor mais rápido
C. Compreensão: começar a programar imediatamente

**24. Associe cada atividade ao que ela faz.**

*Associe cada item da esquerda a um da direita.*

| | |
|---|---|
| 1. Elicitação | A. Revisa a especificação com os usuários antes de construir |
| 2. Negociação | B. Reúne as necessidades das partes interessadas |
| 3. Validação | C. Acompanha cada mudança nos requisitos |
| 4. Gestão | D. Concilia visões conflitantes, custo e risco |

---

**25. Qual modelo mostra melhor os limites de um sistema e quem troca dados com ele?**

A. Diagrama de contexto
B. Máquina de estados
C. Diagrama de classes

**26. Uma coluna que guarda a chave primária de outra tabela para ligar as duas tabelas se chama chave ____.**

*Resposta:* _____

# Gabarito

1. A — Qualquer artefato que faça parte do produto ou de sua documentação pode ser um item de configuração. Os requisitos são um exemplo central.
2. A — Cada clone tem o repositório e o histórico completos, então você pode fazer commit sem conexão.
3. A — Por definição, a diferença entre duas linhas de base é um incremento.
4. B — Falso. O Git é a ferramenta de controle de versões. O GitHub hospeda repositórios Git e acrescenta recursos de colaboração.
5. B — Falso. O git add apenas prepara as mudanças. O git commit as registra.
6. B — Falso. Uma branch é um ponteiro para um commit. O Git não copia os arquivos ao criar uma.
7. A área de preparação, também chamada de índice, contém as mudanças que vão entrar no próximo commit.
8. git switch -c cria e muda de branch. git checkout -b faz o mesmo com a sintaxe antiga.
9. O fetch baixa commits remotos sem mudar sua branch de trabalho. O pull faz o fetch e depois os integra à sua branch atual.
10. Commits pequenos são mais fáceis de revisar, entender e reverter se uma única mudança quebrar algo.
11. 1→B, 2→D, 3→A, 4→C — add prepara, commit registra, status informa e fetch baixa commits remotos sem mudar seus arquivos.
12. 1→B, 2→C, 3→A — A cascata é sequencial; o trabalho iterativo produz linhas de base; os métodos adaptativos fazem ciclos curtos de feedback.
13. A — A edição é preparada pelo git add, então o git status a lista como pronta para commit. Nada entra no histórico até o git commit rodar.
14. A — --oneline mostra um commit por linha, e -3 limita a saída a três commits, do mais recente ao mais antigo.
15. A — Antes de depurar, identifique exatamente quais versões foram entregues. A GCS torna isso rastreável.
16. A — Uma branch de funcionalidade com pull request permite que os revisores vejam o diff e aprovem antes do merge.
17. A — O Git não consegue escolher entre duas edições diferentes das mesmas linhas, então marca o conflito para você resolver.
18. A — Integre primeiro o trabalho remoto. Um push forçado pode apagar os commits de um colega e deve ser evitado em branches compartilhadas.
19. A — O Git guarda o nome e o e-mail do autor em cada commit, então os dois precisam ser configurados primeiro.
20. A — Marcadores não são código válido. Escolha o conteúdo final, remova todos os marcadores, prepare o arquivo e conclua o merge com um commit.
21. A — Ela descreve uma qualidade (velocidade) com uma medida. As outras descrevem comportamentos, então são funcionais.
22. B — Falso. Eles são temporários, necessários apenas para passar da situação antiga para a nova.
23. A — Requisitos que continuam mudando são voláteis. Acordos escritos e um único representante os tornam estáveis.
24. 1→B, 2→D, 3→A, 4→C — A elicitação reúne, a negociação concilia, a validação revisa e a gestão acompanha as mudanças.

25. A — Um diagrama de contexto desenha o sistema como uma caixa, com seus atores externos e fluxos de dados.
26. É uma chave estrangeira. Ela cria relacionamentos como um cliente com muitos pedidos.